Credit Risk Intelligence Platform

An AI-powered platform for loan default prediction, explainability, and natural-language data access

Built by Malavika Krishna, for the NeoStats AI Engineer Internship

Candidate Assignment — every number in this deck comes from a real run against the actual dataset, not a mockup

MY APPROACH

How I thought about this problem

Most take-home submissions on this dataset stop at a single CSV and a model score. I wanted to build something a real credit team could actually use — so I optimized for four things:

Multi-table data

Joined application, bureau, prior-loan, and monthly repayment history — not just a single table

Explainable by design

Every prediction is backed by SHAP reasoning and auditable business rules, not a black box

Talk to the data

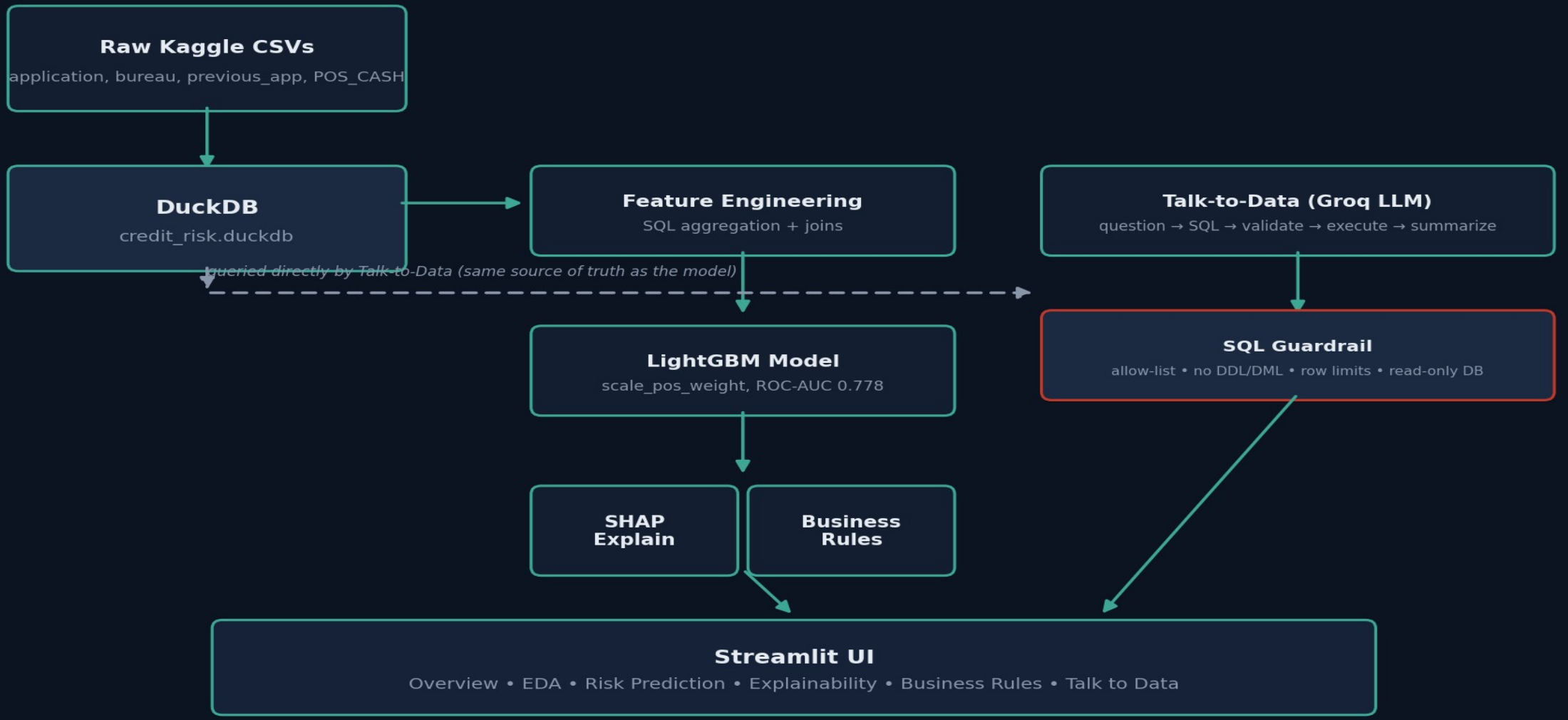
A guardrailed NL-to-SQL chatbot for non-technical stakeholders to explore the data directly

Ship it

Fully Dockerized, single-command deployment

ARCHITECTURE

How the pieces fit together



DATA

Multi-table feature engineering, not just one CSV

Most submissions on this dataset use application_train.csv alone. This platform joins five additional tables — aggregated in SQL for speed on tables up to 27M rows — to capture signal a single-table model would miss.

applications

`307,511 rows`

Core applicant demographics & loan terms

bureau + bureau_balance

`1.7M + 27.3M rows`

External credit history & monthly DPD status

previous_applications

`1.7M rows`

This applicant's prior loans with the same lender

pos_cash_balance

`10M rows`

Monthly days-past-due (DPD) on cash/POS loans

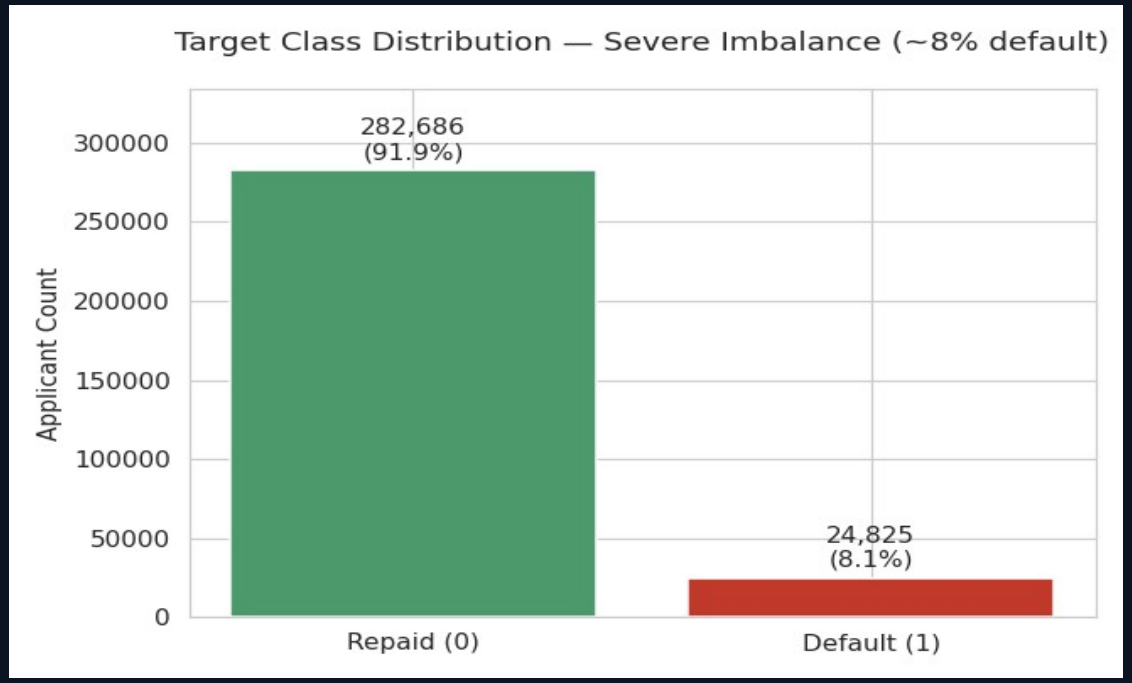
credit_card_balance

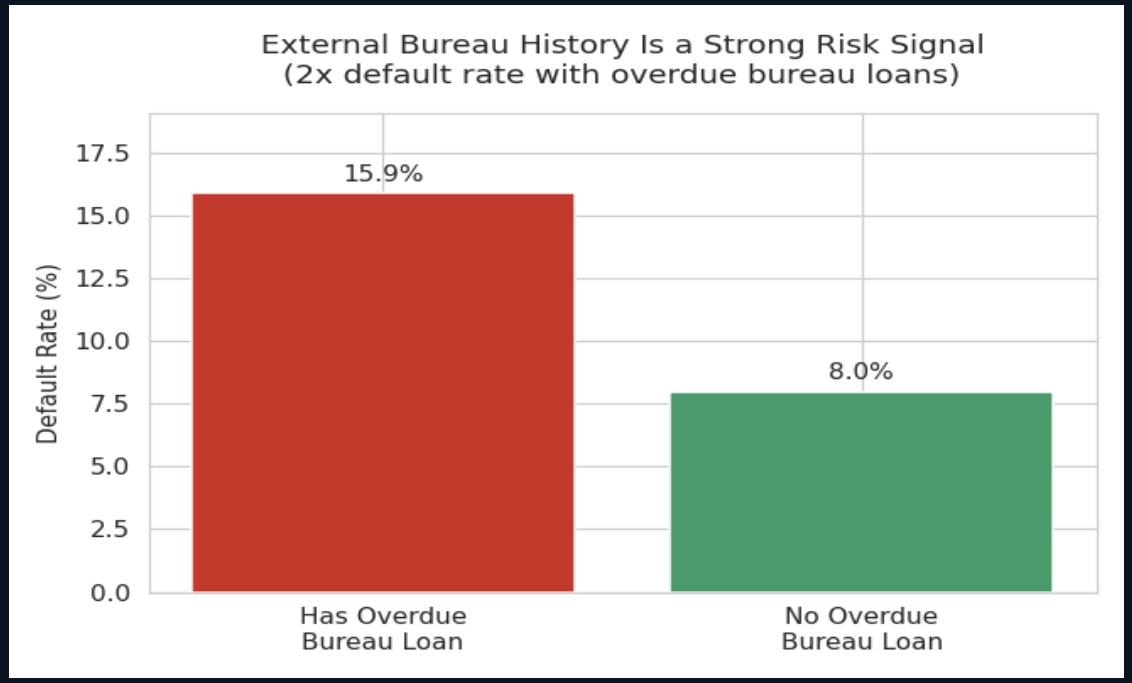
`3.8M rows`

Monthly card balance & utilization — new default-risk signal

Result: 165 features across 6 tables, feeding a single unified model.

What the data says before any model is built





- Only 8.07% of applicants default — a naive "approve everyone" model would be 92% "accurate" and useless
- An overdue external bureau loan nearly doubles default risk (15.9% vs 8.0%)

MACHINE LEARNING

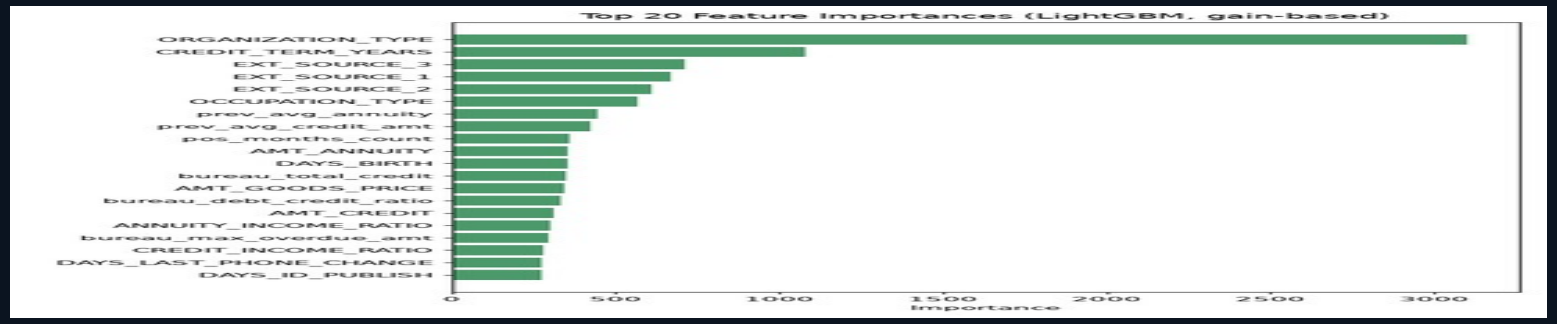
LightGBM with an evidence-based imbalance strategy

ROC-AUC	PR-AUC	Recall @ 0.5	Features
0.780	0.277	66.7%	165

Class imbalance (~8% default rate): scale_pos_weight vs SMOTE — tested, not assumed

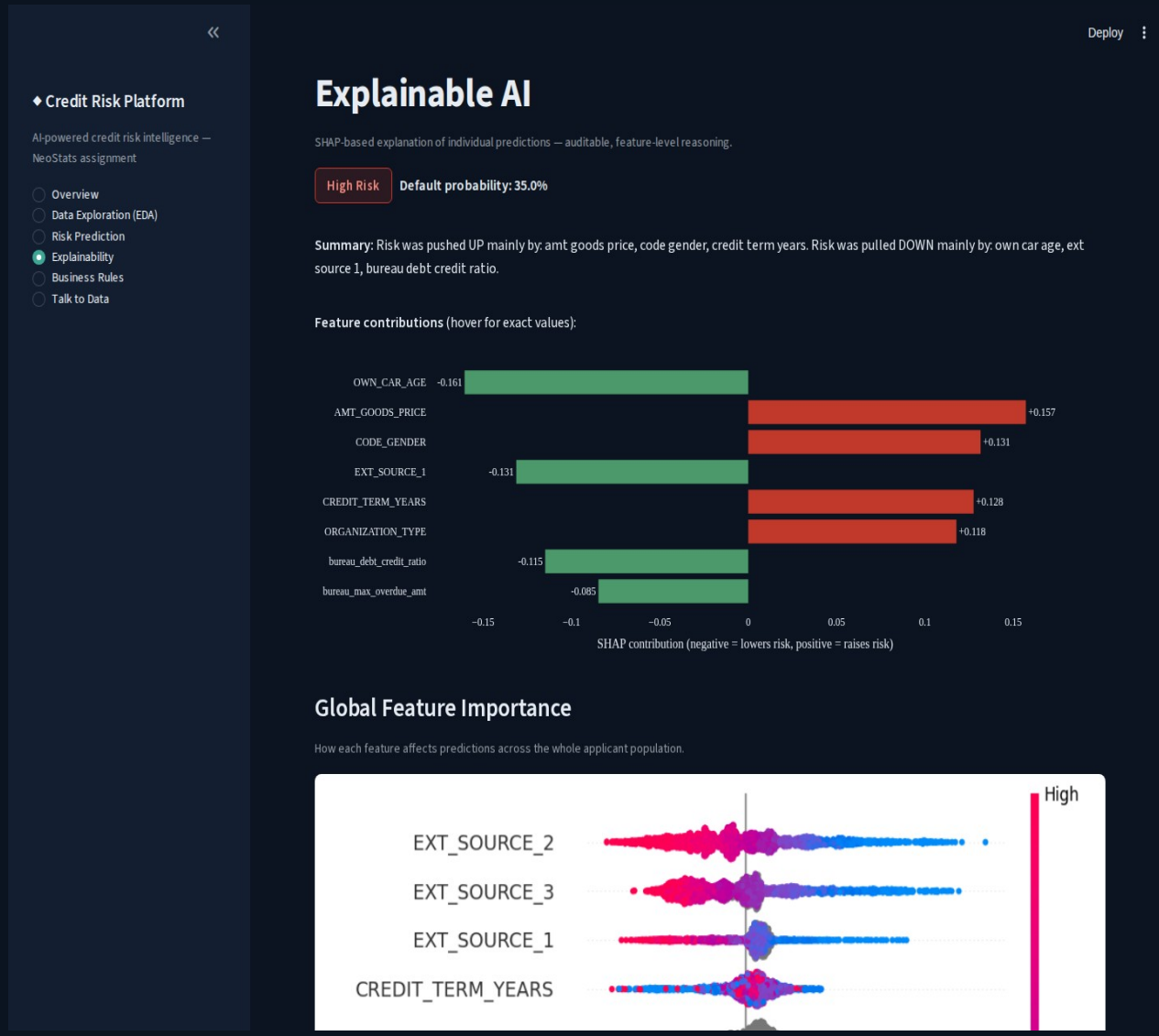
Approach	Validation ROC-AUC	Notes
scale_pos_weight (production)	0.7804	Simpler, faster, no synthetic data
SMOTE (0.5 ratio)	0.7792	Statistically indistinguishable (difference = 0.001)

Evaluated on ROC-AUC / PR-AUC, not accuracy — accuracy is misleading under 8% class imbalance.



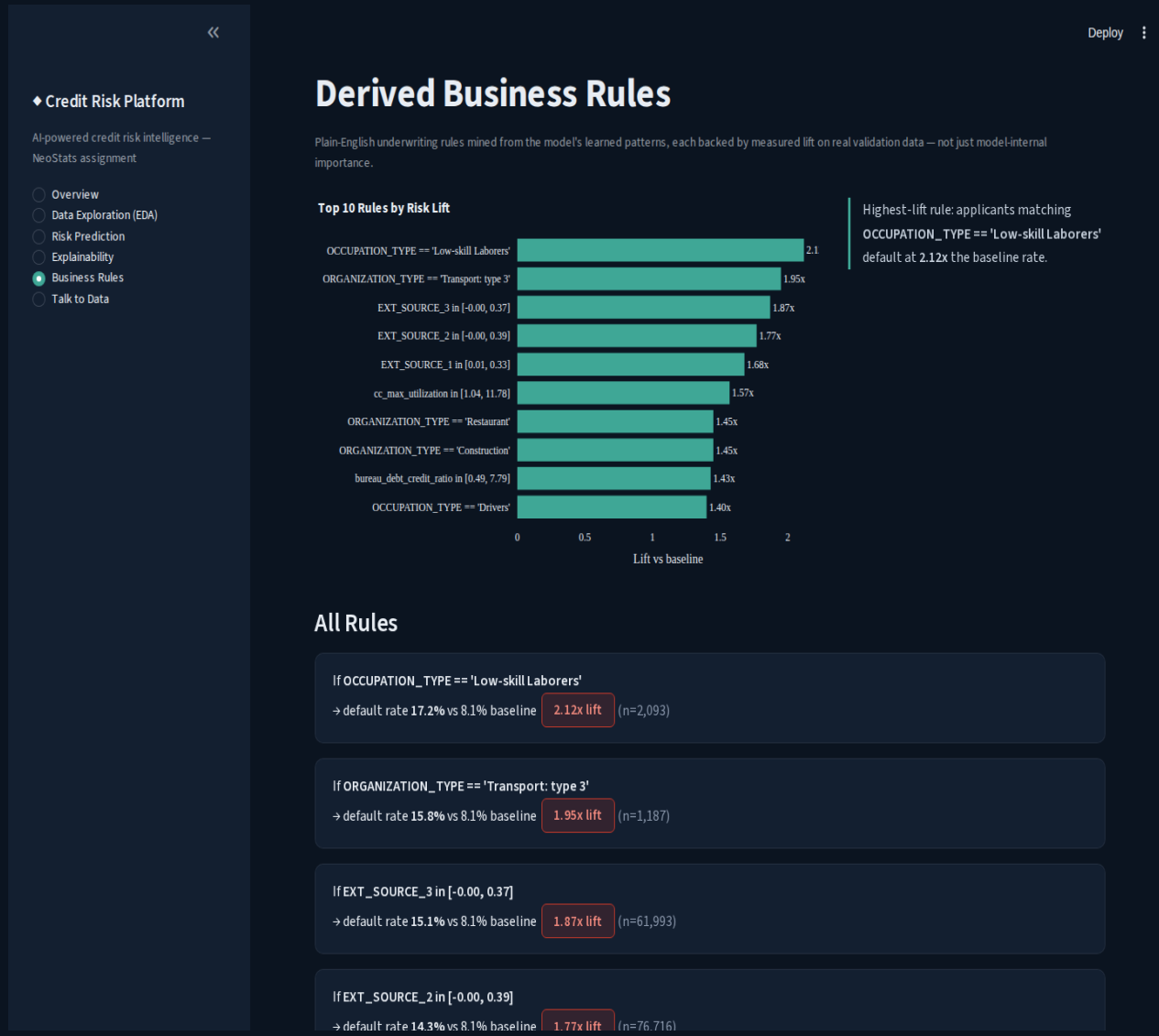
SHAP: auditable reasons behind every prediction

- TreeExplainer — exact SHAP values for gradient-boosted trees, not an approximation
- Interactive diverging bar chart (Plotly) — hover any feature for its exact value and impact
- Per-applicant: top features, direction, and a plain-English summary
- Global: beeswarm plot across a 2,000-applicant sample



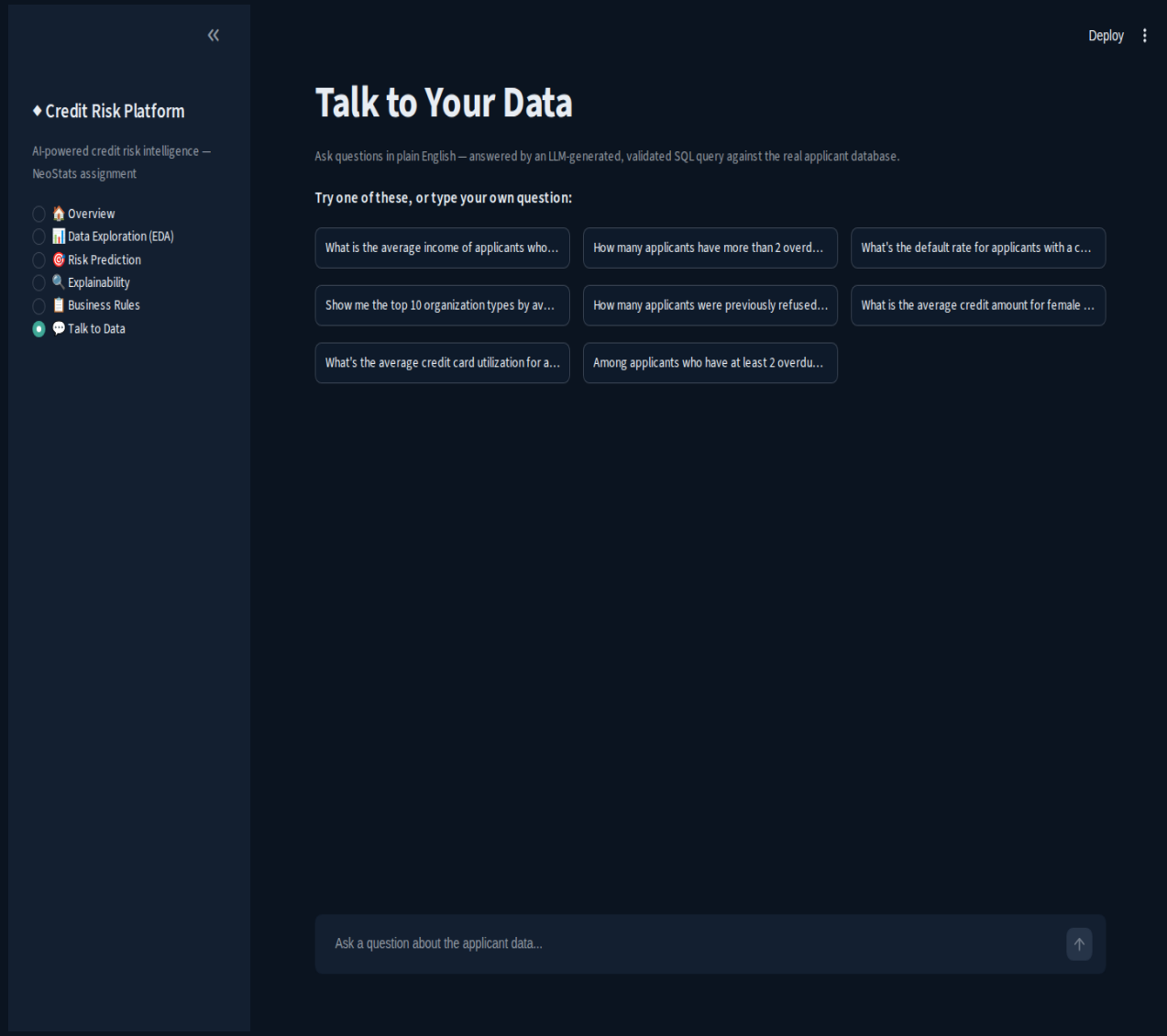
Bridging ML output and credit policy

- Bins top-25 model features and surfaces any bin where default rate is materially above baseline
- Interactive lift chart (Plotly) plus full rule cards below, each with hover-lift animation
- Every rule backed by measured lift + support (n), not just model-internal importance
- Example: Low-skill Laborers default at 2.12x baseline (n=2,093)



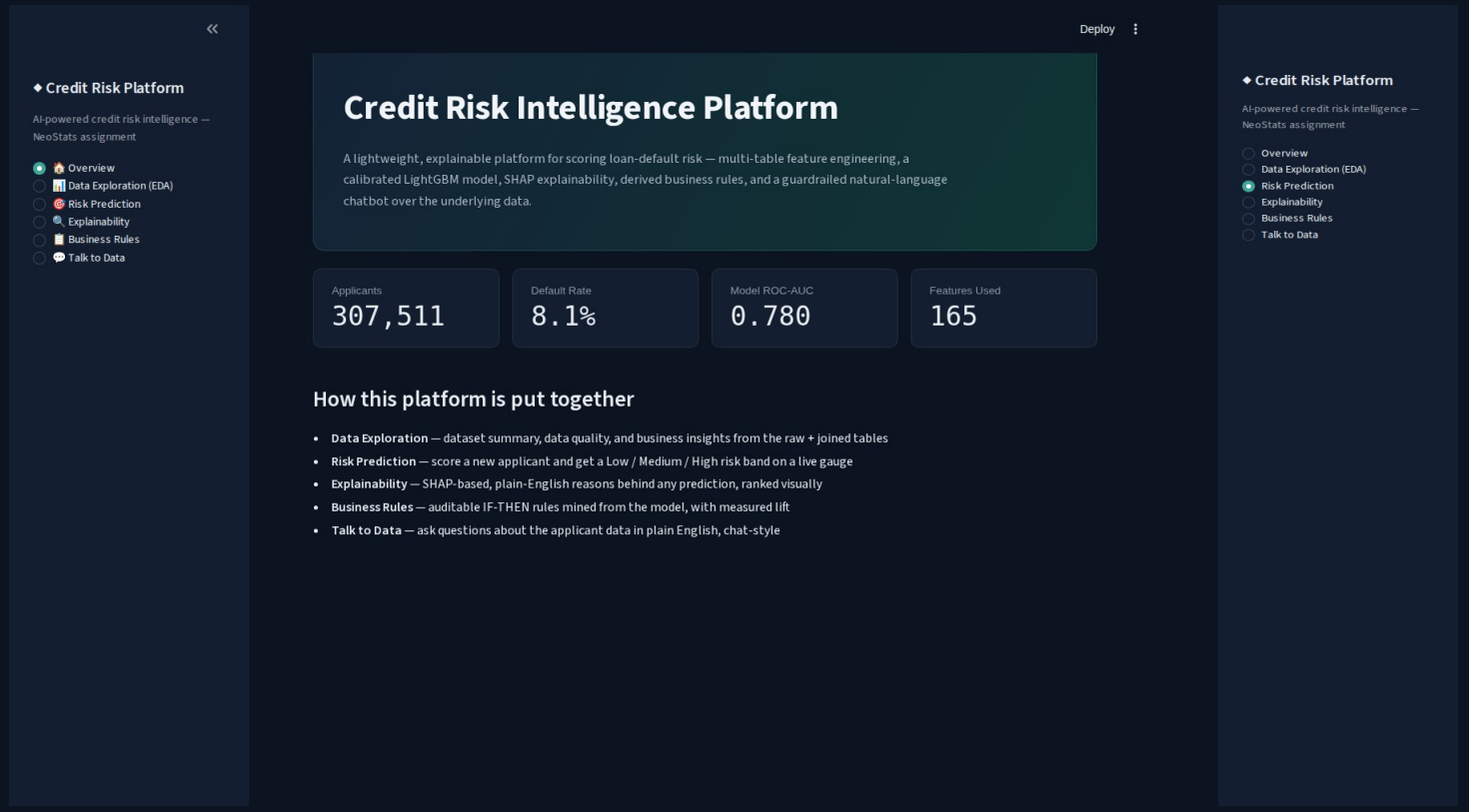
NL-to-SQL with guardrails enforced in code

- Two-stage LLM calls: SQL generation (temp 0.1) → grounded answer summarization from real returned rows
- SQL validator blocks: DDL/DML keywords, chained statements, hallucinated tables/columns
- Auto row-limiting + read-only DB connection as a second defense layer
- Tested against real adversarial inputs (DROP TABLE, injection, fake tables) — all correctly blocked
- 8 working query patterns (exceeds the required 5), incl. a 2-CTE stress test
- 2 real bugs found + fixed via hard questions: CTE aliases misread as tables; REPLACE() vs REPLACE INTO

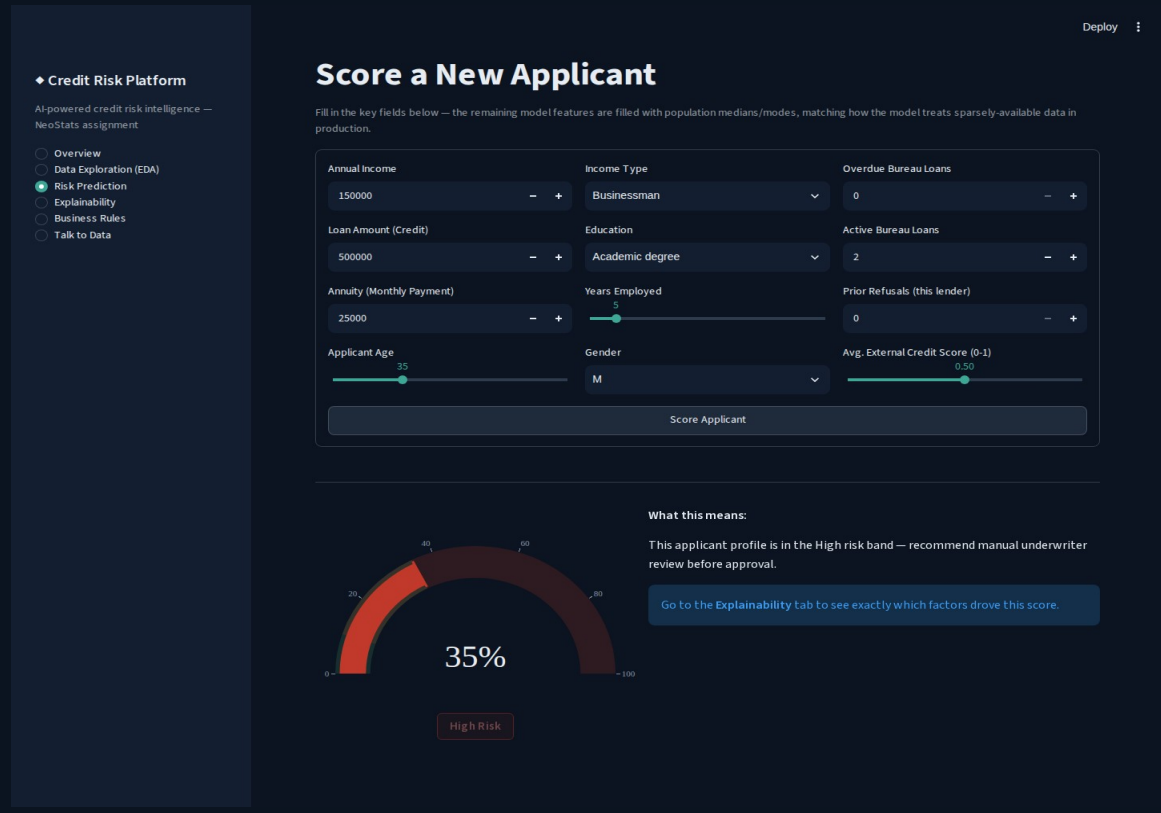


USER INTERFACE

One platform, five sections, real data throughout



Overview (KPIs from real data)



Risk Prediction (live scoring + risk band)

Streamlit • dark navy/teal design system • IBM Plex Sans • interactive Plotly charts • hover animations throughout

DEPLOYMENT

One command to run the full platform

1

Clone & configure

git clone the repo, copy .env.example → .env, add a free Groq API key

2

Prep data & model

Download 5 Kaggle CSVs → run loader, preprocessor, train, evaluate, rules scripts

3

docker-compose up

Builds the image and launches Streamlit on port 8501 — single command

Dockerfile • docker-compose.yml • .env.example — all in the repo root, per the required structure

`github.com/malavika-krishna24/credit-risk-platform`

BEING HONEST WITH YOU

What I'd do next, if I kept going

I'd rather show you exactly where the edges are than pretend this is finished. Here's what I'd tackle first with more time:

installments_payments.csv not joined

Repayment-timing signal is already substantially covered by POS_CASH_balance and credit_card_balance's own DPD tracking

Risk-band thresholds are defaults

Low/Medium/High cutoffs should be tuned against a real bank's approval/loss economics in production

Prediction form uses ~12 key fields

Remaining ~153 features filled from population medians/modes — a deliberate UX tradeoff for a demo interface

No monitoring/retraining pipeline

A production deployment needs drift detection and scheduled retraining — out of scope here

Thank you for reading this far

I genuinely enjoyed building this — the multi-table joins, the SQL guardrails, the SHAP explanations that turned out to match intuition. If you dig into the repo and something looks off or you'd want it done differently, I'd love to talk through the reasoning.

Malavika Krishna

github.com/malavika-krishna24/credit-risk-platform